function main

B1 (entry)
ExprStmt
expr: Call(send_byte) { Nop(Literal) [const:'1'] }

B2 (while.cond)
WhileCond
expr: Nop(Literal) [const:1]

false

true

B3 (after_while)
Nop(exit)

B4 (block)
ExprStmt
expr: Call(send_byte) { Nop(Literal) [const:'2'] }

B40 (block)
ExprStmt
expr: Call(send_byte) { Nop(Literal) [const:'1'] }
ExprStmt
expr: Call(send_byte) { Nop(Literal) [const:'4'] }

B5 (while.cond)
WhileCond
expr: Nop(Literal) [const:2]

false

true

B41 (if.cond)
IfCond
expr: BinaryOp(CompareExpr) { Nop(Identifier) [var:a] | Nop(Literal) [const:3] }

B6 (after_while)
Nop(exit)

B7 (block)
ExprStmt
expr: Call(send_byte) { Nop(Literal) [const:'3'] }

true

false

B42 (block)
ExprStmt
expr: Call(send_byte) { Nop(Literal) [const:'1'] }
ExprStmt
expr: Call(send_byte) { Nop(Literal) [const:'n'] }

B43 (block)
ExprStmt
expr: Call(send_byte) { Nop(Literal) [const:'2'] }
ExprStmt
expr: Call(send_byte) { Nop(Literal) [const:'n'] }

B15 (block)
ExprStmt
expr: Call(send_byte) { Nop(Literal) [const:'5'] }

B8 (if.cond)
IfCond
expr: Nop(Literal) [const:3]

B44 (if.join)
join

B16 (if.cond)
IfCond
expr: Nop(Literal) [const:5]

B10 (if.cond)
IfCond
expr: Nop(Literal) [const:4]

true

false

B9 (block)
ExprStmt
expr: Call(send_byte) { Nop(Literal) [const:'b'] }

B45 (while.cond)
WhileCond
expr: BinaryOp(CompareExpr) { Nop(Identifier) [var:i] | Nop(Literal) [const:10] }

B18 (if.cond)
IfCond
expr: Nop(Literal) [const:6]

B11 (block)
ExprStmt
expr: Call(send_byte) { Nop(Literal) [const:'c'] }

B12 (block)
ExprStmt
expr: Call(send_byte) { Nop(Literal) [const:'l'] }
ExprStmt
expr: Call(send_byte) { Nop(Literal) [const:'4'] }

B17 (block)
ExprStmt
expr: Call(send_byte) { Nop(Literal) [const:'b'] }

B46 (after_while)
Nop(exit)

B47 (if.cond)
IfCond
expr: BinaryOp(CompareExpr) { Nop(Identifier) [var:a] | Nop(Identifier) [var:b] }

B19 (block)
ExprStmt
expr: Call(send_byte) { Nop(Literal) [const:'c'] }

B20 (block)
ExprStmt
expr: Call(send_byte) { Nop(Literal) [const:'6'] }

B13 (if.join)
join

B54 (if.cond)
IfCond
expr: Nop(Literal) [const:1]

B49 (if.cond)
IfCond
expr: BinaryOp(MulExpr) { Nop(Literal) [const:2] | Nop(Literal) [const:3] }

B21 (if.join)
join

B14 (if.join)
join

B48 (block)
Assign(=)
lhs: Nop(Identifier) [var:i]
rhs: BinaryOp(AddExpr) { Nop(Identifier) [var:i] | Nop(Literal) [const:2] }

B55 (if.cond)
IfCond
expr: Nop(Literal) [const:2]

B50 (block)
ExprStmt
expr: Call(send_byte) { Nop(Literal) [const:'b'] }

B51 (block)
ExprStmt
expr: Call(send_byte) { Nop(Literal) [const:'c'] }

B22 (if.join)
join

B62 (if.else)
empty

B52 (if.join)
join

B23 (block)
ExprStmt
expr: Call(send_byte) { Nop(Literal) [const:'7'] }

B56 (if.cond)
IfCond
expr: Nop(Literal) [const:3]

B60 (block)
ExprStmt
expr: Call(z) { }

B57 (block)
ExprStmt
expr: Call(x) { }

B58 (block)
ExprStmt
expr: Call(y) { }

B53 (if.join)
join

B24 (block)
ExprStmt
expr: Call(send_byte) { Nop(Literal) [const:'8'] }

B59 (if.join)
join

B25 (if.cond)
IfCond
expr: Nop(Literal) [const:9]

B61 (if.join)
join

B26 (block)
ExprStmt
expr: Call(send_byte) { Nop(Literal) [const:'b'] }

B27 (if.else)
empty

B28 (if.join)
join

B63 (if.join)
join

B29 (while.cond)
RepeatCond(while)
expr: Nop(Literal) [const:10]

B64 (if.cond)
IfCond
expr: BinaryOp(CompareExpr) { Nop(Identifier) [var:a] | Nop(Literal) [const:3] }

B66 (block)
ExprStmt
expr: Call(send_byte) { Nop(Literal) [const:'2'] }
ExprStmt
expr: Call(send_byte) { Nop(Literal) [const:'n'] }
ExprStmt
expr: Call(send_byte) { Nop(Literal) [const:'2'] }

B30 (after_while)
Nop(exit)

B65 (block)
ExprStmt
expr: Call(send_byte) { Nop(Literal) [const:'1'] }
ExprStmt
expr: Call(send_byte) { Nop(Literal) [const:'n'] }

B67 (block)
ExprStmt
expr: Call(send_byte) { Nop(Literal) [const:'2'] }
ExprStmt
expr: Call(send_byte) { Nop(Literal) [const:'n'] }
ExprStmt
expr: Call(send_byte) { Nop(Literal) [const:'2'] }

B31 (block)
ExprStmt
expr: Call(send_byte) { Nop(Literal) [const:'1'] }
ExprStmt
expr: Call(send_byte) { Nop(Literal) [const:'1'] }

B68 (block)
ExprStmt
expr: Call(send_byte) { Nop(Literal) [const:'2'] }
ExprStmt
expr: Call(send_byte) { Nop(Literal) [const:'2'] }
ExprStmt
expr: Call(send_byte) { Nop(Literal) [const:'n'] }

B32 (if.cond)
IfCond
expr: Nop(Literal) [const:11]

B69 (if.join)
join

B34 (if.cond)
IfCond
expr: Nop(Literal) [const:12]

B33 (block)
ExprStmt
expr: Call(send_byte) { Nop(Literal) [const:'b'] }

B70 (while.cond)
WhileCond
expr: BinaryOp(CompareExpr) { Nop(Identifier) [var:a] | Nop(Literal) [const:3] }

B35 (block)
ExprStmt
expr: Call(send_byte) { Nop(Literal) [const:'c'] }

B36 (block)
ExprStmt
expr: Call(send_byte) { Nop(Literal) [const:'1'] }
ExprStmt
expr: Call(send_byte) { Nop(Literal) [const:'2'] }

B71 (after_while)
Nop(exit)

B72 (block)
ExprStmt
expr: Call(send_byte) { Nop(Literal) [const:'3'] }
ExprStmt
expr: Call(send_byte) { Nop(Literal) [const:'n'] }

B37 (if.join)
join

B73 (block)
ExprStmt
expr: Call(send_byte) { Nop(Literal) [const:'H'] }
ExprStmt
expr: Call(send_byte) { Nop(Literal) [const:'i'] }
ExprStmt
expr: Call(send_byte) { Nop(Literal) [const:'n'] }

B38 (if.join)
join

B0 (exit)
Nop(exit)

B39 (block)
ExprStmt
expr: Call(send_byte) { Nop(Literal) [const:'1'] }
ExprStmt
expr: Call(send_byte) { Nop(Literal) [const:'3'] }

function z

B1 (entry)
ExprStmt
expr: Call(send_byte) { Nop(Literal) [const:'z'] }

B0 (exit)
Nop(exit)

function y

B1 (entry)
ExprStmt
expr: Call(send_byte) { Nop(Literal) [const:'y'] }

B0 (exit)
Nop(exit)

function x

B1 (entry)
ExprStmt
expr: Call(send_byte) { Nop(Literal) [const:'x'] }

B0 (exit)
Nop(exit)